ADVANCED DATABASE SYSTEMS

Implementing Business Logic Using Stored Procedures

Duration	Maximum Marks	Mode	Assessment Type
90 Minutes	30	Open Book	Individual Practical

PART A — STUDENT INSTRUCTIONS

- This is an individual, open-book practical examination.
- You may use course notes, textbooks, SQL reference material, and official DBMS documentation.
- The objective is to implement and demonstrate database-level business logic, not to reproduce SQL syntax from memory.
- You must implement the core business rules inside stored procedure(s). Do not rely only on application-level logic.
- Your solution must include appropriate validation, transaction management, error handling, and testing.
- You must be able to explain your implementation to the evaluator. Code that cannot be explained may not receive full marks.
- Do not manually modify database records to manufacture expected results during testing.
- Use the DBMS-specific syntax and transaction/error-handling mechanisms supported by the laboratory environment.
- The evaluator may change input values or introduce an edge case during demonstration. Your stored procedure should enforce the business rules independently of the test data.
- Save your SQL script and any supporting output/screenshots according to the submission instructions provided by the instructor.

Assessment Philosophy

The assessment follows a professional database-development workflow: Business Requirement → Business Rules → Stored Procedure → Validation → Transaction → Error Handling → Testing → Demonstration.

PART B — BUSINESS CASE

Scenario: E-Commerce Order Processing System

You are working as a Database Developer for an e-commerce platform. The application team requires a database-level operation for placing customer orders. The database must enforce critical business rules so that invalid orders cannot create inconsistent customer balances, inventory, or order records.

Database Tables

Table	Important Columns	Purpose
Customers	CustomerID, CustomerName, Status, CreditLimit, CurrentBalance	Stores customer account and credit information.
Products	ProductID, ProductName, UnitPrice, StockQuantity, Status	Stores product price, inventory, and availability.
Orders	OrderID, CustomerID, OrderDate, TotalAmount, Status	Stores the master record for each order.
OrderItems	OrderItemID, OrderID, ProductID, Quantity, UnitPrice, Subtotal	Stores products and quantities included in an order.

PART C — CODING TASK

Develop a stored procedure named: `CreateCustomerOrder`

The procedure will be called by the application whenever a customer attempts to purchase a product. It must validate the request, enforce all business rules, create the order, update related records, and return a meaningful result.

Required Inputs

Parameter	Description
CustomerID	Customer placing the order
ProductID	Product being purchased
Quantity	Number of units requested

PART D — BUSINESS RULES TO IMPLEMENT

1. Customer Eligibility: The customer must exist and must have Status = ACTIVE. Otherwise, reject the order with a meaningful message.

2. Quantity Validation: Quantity must be greater than zero. Zero or negative quantities must be rejected.

3. Product Eligibility: The product must exist and must have Status = ACTIVE. DISCONTINUED products cannot be ordered.

4. Inventory Validation: Requested quantity must not exceed available StockQuantity.

5. Credit Validation: Calculate ProjectedBalance = CurrentBalance + OrderTotal. The order is allowed only when ProjectedBalance <= CreditLimit.

6. Order Creation: Create a new Orders record and calculate the order total using Quantity × UnitPrice.

7. Order Item Creation: Insert the corresponding OrderItems record using the current product price and calculated subtotal.

8. Inventory Update: Reduce StockQuantity by the quantity ordered.

9. Customer Balance Update: Increase CurrentBalance by the order total.

10. Order Confirmation: After all operations succeed, set the order Status to CONFIRMED.

PART E — TRANSACTION AND ERROR HANDLING

All operations that change the database must be treated as one atomic business transaction. If any operation fails, the database must be returned to the state that existed before the order attempt.

Required principle: No partial order should remain in the database.

- Use the transaction mechanism supported by your DBMS.
- Commit only after all required operations succeed.
- Roll back if a business validation fails after modifications have begun or if an unexpected database error occurs.
- Implement appropriate exception/error handling.
- Return a clear success or failure status and an understandable message.

PART F — REQUIRED TESTING

Demonstrate the test cases.

Test	Input / Condition	Expected Result	Observed Result
1	Valid customer + active product + sufficient stock + credit available	Order successfully created	
2	Blocked/non-existent customer	Order rejected	
3	Insufficient product inventory	Order rejected	
4	Projected balance exceeds credit limit	Order rejected	
5	Quantity is zero or negative	Order rejected	

The evaluator may request an additional test case during demonstration.

PART G — ADVANCED PROFESSIONAL CHALLENGE

Concurrency and Overselling

Assume that only 2 units of a product remain and two customers attempt to purchase those units at nearly the same time. Explain and, where feasible, demonstrate how your implementation prevents both transactions from successfully consuming the same inventory.

The evaluator may ask you to modify the inventory update so that StockQuantity can never become negative, even when multiple transactions execute concurrently.

Discussion Points

- Transaction isolation
- Row-level locking or equivalent DBMS mechanisms
- Atomic conditional inventory updates
- Race conditions
- Consistency of customer balance and stock
- Why checking stock in one statement and updating it later may be unsafe under concurrency
- Stored procedure executes successfully.
- All specified business rules are implemented.
- Transaction and rollback behavior is demonstrated.
- At least five test cases are documented.
- Failure cases are verified, not merely described.
- The student can explain each major section of the procedure.
- SQL script/output is saved according to the instructor's submission instructions.